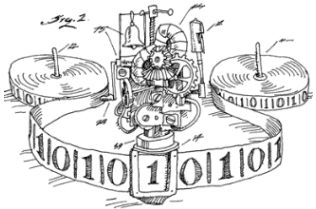
Teoría de la Computación y Lenguajes Formales



Teoría de la Computación y Lenguajes Formales: Una introducción hacia la Automatización de Procesos

Édgar Chacón Ramírez
Departamento de Computación
Semestre 2026 - B

Alcance de la sesión 1

UNIVERSIDAD DE LOS ANDES VENEZUELA

Presentación

Motivación al curso

Formalismos de computación

Gramáticas Formales
- Gramáticas Regulares (Tipo 3)
- Gramáticas Libres de Contexto

Conclusión

Responsable del curso: Édgar Chacón.
Referencia: https://orcid.org/0000-0001-5612-0368
Area de Interés: Automatización industrial. Sistemas Holónicos de Producción. Industria 4.0

Libros texto y referencias

UNIVERSIDAD DE LOS ANDES VENEZUELA

Libros texto

- Jurado Málaga, E. (2008). *Teoría de autómatas y lenguajes formales*. Badajoz: Universidad de Extremadura. Servicio de Publicaciones. [Disponible en http://dehesa.unex.es/handle/10662/2367]
- Moral, S. (2017). *Teoría de Autómatas y Lenguajes Formales*. Granada: Universidad de Granada [Disponible en http://www.academia.edu/download/52014903/automata.pdf]

Recomendado.
Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007)
Teoría de autómatas, lenguajes y computación
Madrid: Pearson (3ª ed.)



Libros texto y referencias

UNIVERSIDAD DE LOS ANDES VENEZUELA

Videos de la Prof. Hilda Yelitza Contreras Zambrano

Colección de videos elaborados por la Profesora para el curso.
www.youtube.com/@hildayelitzacontreraszambr1261

Material complementario

Sipser, M. (2006). *Introduction to the Theory of Computation*. Boston, MA: Thomson Course Technology [Disponible en https://fuuu.be/polytech/INFOF408/Introduction-To-The-Theory-Of-Computation-Michael-Sipser.pdf]

El por qué de un curso de Teoría de la Computación

UNIVERSIDAD DE LOS ANDES VENEZUELA

Qué estudia la teoría de la computación

La Teoría computacional es matemática y abstracta en espíritu, pero su motivación se basa en el uso de una capacidad de resolución de problemas reales.
Wikipedia: La teoría de la computación o teoría de la informática es una rama de las ciencias de la computación que estudia los principios fundamentales de la computación desde un enfoque abstracto y formal. Su objetivo principal es entender qué problemas pueden resolverse mediante procedimientos computacionales, cuánta memoria y tiempo se requiere para ello, y cómo representar formalmente dichos procesos.
Debe responder a:

- a) Qué se puede calcular, b) Qué se puede automatizar
  Teoría de la Computabilidad (Lenguajes Formales y Autómatas)
- Cuantos recursos son necesarios
  Teoría de la Complejidad

El por qué de un curso de Teoría de la Computación

UNIVERSIDAD DE LOS ANDES VENEZUELA

Idea central

La teoría no son solo abstracciones; es el fundamento de todo sistema informático. Define los límites de lo que podemos y no podemos automatizar.

Aplicaciones de la Teoría de la Computación

Control Automático: Un sistema de control es, en esencia, un autómata que lee entradas (sensores) y produce salidas (actuadores) según reglas predefinidas.

Gestión de Operaciones: La planificación y logística se modelan como problemas de optimización en grafos (Autómatas, Gramáticas) y verificación de procesos (Máquinas de Turing).

Flujos de Trabajo: Un flujo de trabajo es un "programa" para humanos y sistemas. La teoría nos ayuda a analizar sus propiedades (¿siempre termina?, ¿es correcto?).

Aplicaciones de la Teoría de la Computación

UNIVERSIDAD DE LOS ANDES VENEZUELA

Desarrollo de una "Máquina" para automatizar un proceso

Pasos:

1. Entender el proceso. Conocer cual es el comportamiento del proceso. Está asociado al término Dinámica
2. Describirlo de manera formal, para poder analizarlo. Implica tener una manera precisa que indique como se comporta.

$$\dot{x} = f(x, u)$$

Aplicaciones de la Teoría de la Computación

UNIVERSIDAD DE LOS ANDES VENEZUELA

Desarrollo de una "Máquina" para automatizar un proceso

Pasos:

1. Entender el proceso. Conocer cual es el comportamiento del proceso. Está asociado al término Dinámica
2. Describirlo de manera formal, para poder analizarlo. Implica tener una manera precisa que indique como se comporta.

$$\dot{x} = f(x, u)$$

o

$$x_{k+1} = f(x_k, u_x)$$

3. Construir la máquina que va a tener el comportamiento deseado o que va a asegurar que evolucione como se espera.

Criterios para la construcción de la Máquina

1. El problema es decidible. Si no lo es, no podemos construir una máquina que lo resuelva.
2. El problema es computable. Se debe resolver en tiempo finito, y con un uso de recursos acotado.

Aplicaciones de la Teoría de la Computación

UNIVERSIDAD DE LOS ANDES VENEZUELA

Criterios para la construcción de la Máquina

1. El problema es decidible. Si no lo es, no podemos construir una máquina que lo resuelva.
2. El problema es computable. Se debe resolver en tiempo finito, y con un uso de recursos acotado.

Herramientas

- Lenguajes formales, nos permiten describir el problema de manera que una máquina lo entienda.
- Generación de instrucciones para que otra máquina ejecute las tareas, o haga ejecutar las tareas.

UNIVERSIDAD DE LOS ANDES VENEZUELA

Formalismos de Computación

Inicio de los formalismos en computación

UNIVERSIDAD DE LOS ANDES VENEZUELA

Hilbert y la teoría de Computabilidad



Hilbert (1928): Tiene el propósito de crear un sistema matemático formal "completo" y "consistente", en el que todas las aseveraciones pudieran plantearse con precisión. (Lógica de predicados)

Principios que se debían cumplir eran:

1. El sistema debía ser consistente; es decir, no debía existir un enunciado P tal que P y su negación fueran simultáneamente demostrables a partir de los axiomas.
2. La validez de cualquier demostración basada en esos axiomas debía ser verificable algorítmicamente en una cantidad finita de pasos.
3. Dado cualquier enunciado P, o bien él o bien su negación debía ser demostrable a partir de los axiomas.
4. La consistencia de los axiomas (es decir, la validez de la primera condición) debía ser verificable algorítmicamente en una cantidad finita de pasos.

Inicio de los formalismos en computación

La propuesta de Hilbert fue refutada por varios matemáticos que trabajaban en el área de la Lógica Matemática.

El aporte de los lógicos

Kurt Gödel establece en 1929

> *En una lógica de primer orden, toda fórmula que es válida en un sentido lógico es demostrable.*

Existe un número finito de pasos para alcanzar el resultado.

Respecto al planteamiento de Hilbert escribe:

> *PRIMER TEOREMA: Si se cumplen las dos primeras condiciones planteadas por Hilbert entonces la tercera nunca podrá cumplirse.*

Inicio de los formalismos en computación

UNIVERSIDAD DE LOS ANDES VENEZUELA

El aporte de los lógicos

Alonzo Church. La noción de calculabilidad. Cálculo λ

Se define como todo tratamiento sistemático realizado por un proceso físico o mecánico puede expresarse por un conjunto de reglas de cálculo, que pueden ser definidas de manera distinta y demostrar que matemáticamente son equivalentes.

Inicio de los formalismos en computación

UNIVERSIDAD DE LOS ANDES VENEZUELA

Calculabilidad

Alain Turing

> *La máquina de Turing consta de un cabezal lector/escritor y una cinta infinita en la que el cabezal lee el contenido, borra el contenido anterior y escribe un nuevo valor. Las operaciones que se pueden realizar en esta máquina se limitan a: Mover el cabezal lector/escritor hacia la derecha.*
>
> *Las MCLs [máquinas de computación lógica: la expresión de Turing para las máquinas de Turing] pueden hacer cualquier cosa que podamos describir como [un procedimiento] "puramente mecánico" 1948.*

Los aportes de Turing y Church son muy próximos y realizados en la misma época aunque se desconocían entre ellos.

Tesis de Church – Turing

Dado un Método o Procedimiento **M**

1. **M** es expresado en términos de un número finito de instrucciones exactas (cada instrucción es expresada por medio de un número finito de símbolos);
2. **M** debe, si no se produce ningún error, producir siempre el resultado deseado en un número finito de pasos;
3. **M** puede (en la práctica, o en principio) llevarse a cabo por un ser humano sin la ayuda de ninguna máquina, simplemente con papel y lápiz;
4. **M** no requiere ningún tipo de comprensión ni de ingenio por parte del humano que intenta resolverlo.

Información complementaria video https://www.youtube.com/watch?v=FS73NykTfMk

Problemas indecidibles

UNIVERSIDAD DE LOS ANDES VENEZUELA

Definición

No todos los problemas pueden ser resueltos.
En teoría de la computabilidad y en teoría de la complejidad computacional, un problema indecidible es un problema de decisión para el cual es imposible construir un algoritmo que siempre conduzca a una respuesta de sí o no correcta.
Ejemplos de problemas indecidibles:

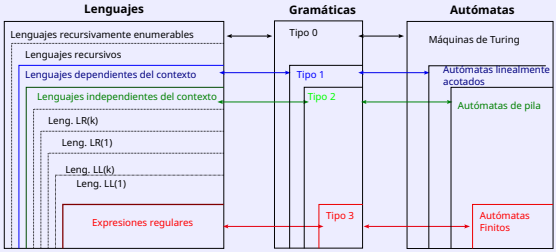
- La propuesta de Hilbert
- Inferencia y verificación de tipos en lógica de segundo orden.
- El problema de la parada o problema de la detención para máquinas de Turing.

Gramáticas Formales

Jerarquía de Chomsky

UNIVERSIDAD DE LOS ANDES VENEZUELA

Relación Autómatas – Gramáticas – Lenguajes



Tomado de Jurado Málaga, 2008

Tipos de Gramáticas

Tipo	Nombre	Autómata
3	Regular	AF
2	Libre contexto	AP
1	Sens. contexto	MT acotada
0	Sin restricción	MT

Nota

Cada clase de lenguaje **contiene** a la anterior: Tipo 3 $\subset$ Tipo 2 $\subset$ Tipo 1 $\subset$ Tipo 0.

Gramáticas Regulares (Tipo 3)

UNIVERSIDAD DE LOS ANDES VENEZUELA

Forma de las producciones

Son las más restrictivas. Existen dos variantes, y una gramática debe usar **una sola** de ellas de forma consistente:

$$\text{Lineal por la derecha:} \quad A \mapsto aB \mid a \mid \varepsilon$$

$$\text{Lineal por la izquierda:} \quad A \mapsto Ba \mid a \mid \varepsilon$$

con $A, B \in V_n$ y $a \in V_t$.

Propiedad estructural

Cada producción introduce **a lo sumo un** no terminal, y este siempre aparece en el mismo extremo de la cadena (derecho o izquierdo, nunca mezclados). No hay contexto que condicione la reescritura.

Equivalencia computacional

Los lenguajes Tipo 3 son exactamente los reconocidos por un **autómata finito** (determinista o no determinista) y son descritos por **expresiones regulares**.

Gramáticas Regulares – Ejemplo

UNIVERSIDAD DE LOS ANDES VENEZUELA

Gramática

$G = (\{S, A\},\ \{0, 1\},\ P,\ S)$

$P = \{$

$S \mapsto 0A \mid 1S,$

$A \mapsto 0A \mid 1A \mid \varepsilon$

$\}$

Derivación de ejemplo

$$S \mapsto 1S \mapsto 10A \mapsto 101A \mapsto 101$$

El lenguaje generado es

$$L(G) = 1^* \, 0 \, \{0, 1\}^*$$

es decir, toda cadena que contiene al menos un 0.

Nota

Cada paso agrega un símbolo terminal por el mismo extremo; el no terminal "viaja" hacia el otro lado hasta desaparecer.

De la gramática al workflow: Tipo 3

UNIVERSIDAD DE LOS ANDES VENEZUELA

Idea

Un autómata finito es exactamente el modelo correcto cuando el **siguiente paso válido depende solo del estado actual**, sin memoria de lo que pasó antes.

Caso: ciclo de una máquina en una PyME

$$\text{Ocioso} \xrightarrow{\text{setup}} \text{Preparando} \xrightarrow{\text{iniciar}} \text{Produciendo} \xrightarrow{\text{fin}} \text{Ocioso}$$

Cada transición corresponde a un evento del piso de planta. El lenguaje $L(G)$ es el conjunto de **bitácoras de eventos válidas** de esa máquina.

Pregunta de validación

Dada una secuencia de eventos registrada por el sistema (por ejemplo, en FÉNIX), ¿pertenece a $L(G)$? Este es el problema de **pertenencia**, y para gramáticas Tipo 3 se resuelve en tiempo lineal: basta con recorrer el autómata evento por evento.

Gramáticas Libres de Contexto (Tipo 2)

UNIVERSIDAD DE LOS ANDES VENEZUELA

Forma de las producciones

Toda producción tiene la forma

$$A \mapsto \alpha \qquad \text{con } A \in V_n,\ \alpha \in (V_n \cup V_t)^*$$

El lado izquierdo es **un único** no terminal: se reescribe *sin importar* lo que lo rodea.

Propiedad estructural

A diferencia de las Tipo 1, aquí A se puede sustituir en **cualquier** contexto en que aparezca. Permiten recursión y anidamiento, por lo que capturan estructuras balanceadas (paréntesis, bloques anidados, expresiones aritméticas).

Equivalencia computacional

Los lenguajes Tipo 2 son exactamente los reconocidos por un **autómata de pila** (pushdown automaton).

Gramáticas Libres de Contexto – Ejemplo

UNIVERSIDAD DE LOS ANDES VENEZUELA

Gramática

$G = (\{S\},\ \{a, b\},\ P,\ S)$

$P = \{$

$\quad S \mapsto aSb \mid \varepsilon$

$\}$

Derivación de ejemplo

$$S \mapsto aSb \mapsto aaSbb \mapsto aabb$$

El lenguaje generado es

$$L(G) = \{a^n b^n \mid n \geq 0\}$$

Contraste con Tipo 1

Note que $L = \{a^n b^n\}$ es libre de contexto, mientras que $L = \{a^n b^n c^n\}$ (visto antes) exige el contexto adicional de una gramática Tipo 1: la CFG no alcanza para "recordar" tres conteos simultáneos.

De la gramática al workflow: Tipo 2

UNIVERSIDAD DE LOS ANDES VENEZUELA

Idea

Cuando un proceso puede **contener** otros procesos que deben abrirse y cerrarse antes de continuar, ya no basta un autómata finito: se necesita una pila.

Caso: arquitectura holónica de FÉNIX

Un holón puede estar compuesto de sub-holones, cada uno con su propio ciclo de apertura y cierre. La gramática

$$S \mapsto (\,S\,) \mid S\,S \mid \varepsilon$$

genera exactamente las estructuras de holones **bien anidadas**: todo sub-proceso abierto se cierra antes de que cierre su contenedor.

Pregunta de validación

¿Está bien formada la jerarquía de un proceso dado – por ejemplo, DIS-MOL-DIL descompuesto en sub-holones – o hay un sub-proceso que quedó abierto? Es el mismo problema que verificar paréntesis balanceados, y se resuelve con una pila: el autómata de pila **es** el verificador.

De la gramática al workflow: Tipo 1

UNIVERSIDAD DE LOS ANDES VENEZUELA

Idea

Hay pasos cuya validez **no depende solo del propio proceso**, sino del estado de los procesos **vecinos** que comparten un recurso. Ya no basta con una pila: hace falta memoria del contexto alrededor.

Caso: adquisición/liberación de equipo en DIS-MOL-DIL

Una etapa (por ejemplo, Molienda) solo puede **tomar** un equipo compartido cuando la etapa vecina (Dispersión) lo ha **liberado** *y* la etapa siguiente (Dilución) está en condición de recibirlo. La regla de reescritura tiene la forma

$$\alpha\, A\, \beta \;\mapsto\; \alpha\, \gamma\, \beta$$

donde α y β representan el estado de las etapas vecinas: la transición del equipo solo es válida *en ese contexto*, no en cualquier momento.

Pregunta de validación

¿Es válida esta adquisición de equipo dado el estado actual de sus vecinas? Aquí la verificación deja de ser local: no basta con mirar el paso aislado ni con una pila – hay que mirar el contexto completo. Es el costo de modelar recursos compartidos con precisión.

Nota

Este es un boceto general de la restricción; ajustar el ejemplo en cuanto se precise si la regla exacta de DIS-MOL-DIL es de tipo mutex/semáforo o de conteos que deben coincidir (como $a^n b^n c^n$).

De la gramática al workflow: Concurrencia y Handshake

UNIVERSIDAD DE LOS ANDES VENEZUELA

Idea

En los sistemas reales (ERP o Planta), los procesos no se ejecutan aislados: operan en **paralelo** y coordinan sus acciones mediante **sincronización síncrona (handshake)**.

Caso: Coordinación entre procesos independientes

Dos procesos locales ejecutan sus propios autómatas:

$$P_1(\text{Ventas/Alimentador}): \quad \text{Listo} \xrightarrow{!req} \text{Espera} \xrightarrow{?ack} \text{Listo}$$

$$P_2(\text{Finanzas/Horno CNC}): \quad \text{Libre} \xrightarrow{?req} \text{Procesando} \xrightarrow{!ack} \text{Libre}$$

El **handshake** ocurre cuando la emisión $!req$ de P_1 y la recepción $?req$ de P_2 coinciden simultáneamente, produciendo una acción síncrona interna (τ).

Propiedad de Coordinación

El comportamiento global del sistema compuesto $P_1 \parallel P_2$ no es la simple suma de los dos procesos, sino su **producto síncrono**. Esto permite analizar formalmente si existen riesgos de **bloqueo mutuo (deadlock)**.

Gramáticas No Restringidas (Tipo 0)

UNIVERSIDAD DE LOS ANDES VENEZUELA

Forma de las producciones

Toda producción cumple simplemente

$$\alpha \mapsto \beta \qquad \text{con } \alpha \neq \varepsilon,\ \alpha, \beta \in (V_n \cup V_t)^*$$

Sin ninguna restricción sobre el contexto ni sobre las longitudes relativas de α y β.

Propiedad clave

A diferencia de las Tipo 1, aquí **sí** se permite que $|\beta| < |\alpha|$: la cadena puede *acortarse* en una derivación. Se pierde la garantía de que el proceso de derivación termine.

Equivalencia computacional

Los lenguajes Tipo 0 son exactamente los reconocidos por una **Máquina de Turing** completa. El problema de pertenencia ($w \in L(G)$?) es, en general, **indecidible** – el mismo problema de la parada visto al inicio de la sesión.

Gramáticas No Restringidas – Ejemplo

UNIVERSIDAD DE LOS ANDES VENEZUELA

Producción distintiva

$G = (\{S, A, B\},\ \{a, b\},\ P,\ S)$

$P = \{$
$\quad S \mapsto ASB \mid \varepsilon,$
$\quad AB \mapsto \varepsilon$
$\}$

Derivación de ejemplo

$$S \mapsto ASB \mapsto AB \mapsto \varepsilon$$

La producción $AB \mapsto \varepsilon$ **reduce** la longitud de la cadena de 2 a 0.

Contraste con Tipo 1

Esta producción sería inválida en una gramática Tipo 1, donde se exige $|\alpha A \beta| \leq |\alpha \gamma \beta|$. Es precisamente esta libertad de acortar la que rompe la garantía de terminación.

De la gramática al workflow: Tipo 0

UNIVERSIDAD DE LOS ANDES VENEZUELA

Idea

Un motor de reglas de negocio completamente genérico – sin restricción estructural sobre qué puede reescribir cada regla – tiene el poder de una máquina de Turing. Se gana expresividad total, pero se pierde toda garantía de verificación.

Caso: motor de reglas sin restricciones estructurales

Muchos sistemas BPM permiten reglas arbitrarias: condicionales anidados, llamadas recursivas a otros procesos, modificación dinámica del propio flujo en tiempo de ejecución. Formalmente, esto equivale a permitir cualquier producción $\alpha \mapsto \beta$: no hay estructura que garantice que el proceso termine.

Pregunta de validación

¿Terminará este proceso alguna vez? Para gramáticas Tipo 0 esta pregunta es, en general, **indecidible** – el mismo problema de la parada de las Máquinas de Turing. Es la razón formal por la que los motores de workflow reales (BPMN, motores de reglas de negocio) imponen restricciones estructurales deliberadas: sacrifican expresividad para recuperar la capacidad de verificar el proceso *antes* de ejecutarlo.

Conclusión

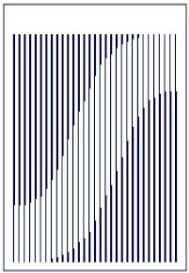
Un proceso del mundo real tiene comportamiento describible.

- Ese comportamiento tiene una gramática
- El comportamiento puede validarse, simularse
- La gramática puede ejecutarse mediante una máquina que la interprete

¡Muchas gracias!



UNIVERSIDAD
DE LOS ANDES
VENEZUELA



Édgar Chacón Ramírez
email: echacon@ula.ve

Jurado Málaga, E. (2008). *Teoría de autómatas y lenguajes formales*. Universidad de Extremadura. Servicio de Publicaciones.